Primeros pasos con Git y GitHub

> Parte de este contenido se ha generado con ayuda de inteligencia artificial. Revísalo antes de usarlo.

Este curso introduce los conceptos fundamentales del control de versiones utilizando Git y la plataforma colaborativa GitHub. Está diseñado específicamente para personas que se inician en la programación y desean aprender a gestionar sus proyectos de forma ordenada y profesional. Recuerde que este material es de carácter formativo general y está orientado al autoaprendizaje básico.

Objetivos

- Identificar la utilidad del control de versiones en el desarrollo de software.
- Instalar y configurar Git en un entorno de desarrollo local.
- Crear y gestionar repositorios locales y remotos.
- Realizar operaciones básicas como commit, push y pull.
- Colaborar en proyectos sencillos usando GitHub.

Módulo 1: Introducción al Control de Versiones y Git

Comprende qué es el control de versiones y aprende a instalar y configurar Git en tu computadora.

¿Qué es Git y por qué necesitas un sistema de control de versiones? (15 min)

Introducción al control de versiones

Cuando comenzamos a escribir código o a crear documentos, es muy común guardar múltiples copias con nombres como *proyecto_final.txt*, *proyecto_final_v2.txt* o *proyecto_definitivo_real.txt*. Este método manual es desordenado, propenso a errores y dificulta saber qué cambios se hicieron y cuándo. Un sistema de control de versiones soluciona este problema registrando cada modificación realizada en los archivos a lo largo del tiempo.

Nota: Este material formativo general está diseñado con fines educativos y proporciona bases prácticas para principiantes.

¿Qué es Git y cómo funciona?

Git es un sistema de control de versiones distribuido, creado originalmente por Linus Torvalds. A diferencia de otros sistemas centralizados, Git permite que cada desarrollador tenga una copia completa del historial del proyecto en su propia computadora. Esto significa que puedes trabajar sin conexión a internet y realizar operaciones complejas de manera local con gran velocidad.

Git no guarda una copia completa de todos los archivos cada vez que haces un cambio, sino que toma una serie de instantáneas (snapshots) del estado de tu proyecto en momentos específicos.

Ejemplo práctico de la utilidad de Git

Imagina que estás escribiendo un programa y añades una nueva función que rompe por completo el funcionamiento del código. Sin Git, tendrías que recordar exactamente qué líneas modificaste para borrarlas o buscar la copia anterior. Con Git, puedes comparar el estado actual con el de la última versión estable (un `commit` previo) y revertir los cambios defectuosos en cuestión de segundos, ahorrando tiempo y frustración.

Ideas clave de la lección

Los puntos más importantes que debes recordar son:

- El control de versiones evita la pérdida de información y el caos de nombrar archivos con sufijos repetitivos.
- Git es un sistema distribuido que guarda instantáneas de tu proyecto.
- Te permite viajar en el tiempo a través del historial de tu código para recuperar versiones anteriores o corregir errores rápidamente.

Instalación y configuración inicial de Git (15 min)

Preparando tu entorno de desarrollo

Antes de poder utilizar Git en tus proyectos, es necesario instalarlo en tu computadora y realizar una configuración básica de identidad. Esta configuración sirve para que Git sepa quién está realizando cada cambio en el código, lo cual es fundamental cuando trabajas de forma individual o en equipo.

Pasos para la instalación de Git

La instalación de Git varía ligeramente según tu sistema operativo:

- **En Windows:** Debes descargar el instalador oficial desde el sitio web de Git y seguir las instrucciones del asistente, manteniendo por defecto las opciones recomendadas.
- **En macOS:** Puedes instalarlo mediante las herramientas de desarrollo de Xcode o a través de administradores de paquetes recomendados.
- **En Linux:** Se instala fácilmente utilizando el gestor de paquetes de tu distribución (por ejemplo, `sudo apt install git` en Ubuntu).

Configuración de tu identidad en Git

Una vez instalado, abre tu terminal o consola de comandos y configura tu nombre de usuario y correo electrónico ejecutando los siguientes comandos básicos:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_correo@ejemplo.com"
```

Estos datos quedarán asociados a cada una de las contribuciones que realices utilizando Git.

Resumen de la instalación

Has aprendido que instalar Git es un proceso rápido y directo en cualquier sistema operativo. La configuración del nombre y el correo electrónico mediante la terminal es un paso obligatorio y único que asegura la correcta identificación de tu autoría en el código.

Actividades

Verificación y configuración del entorno Git

Abre la terminal o línea de comandos de tu computadora y realiza los siguientes pasos:

1. Verifica que Git se haya instalado correctamente escribiendo el comando `git --version` y observando que devuelva un número de versión.
2. Configura tu nombre y tu dirección de correo electrónico utilizando los comandos `git config --global user.name` y `git config --global user.email` con tus datos reales.
3. Comprueba que la configuración se guardó correctamente ejecutando el comando `git config --list` en tu terminal.

Evaluación: Comprueba lo aprendido: Introducción al Control de Versiones y Git

Para aprobar: 60 %

Pregunta 1 (1 punto)

¿Cuál es la principal característica de Git como sistema de control de versiones?

- [ ] Es un sistema centralizado que depende de un único servidor principal.
- [x] Es un sistema distribuido donde cada usuario posee el historial completo del proyecto.
- [ ] Solo funciona si estás conectado a una red local o internet.

Explicación. Git es un sistema distribuido, lo que significa que cada usuario tiene una copia completa del historial del proyecto en su propia computadora.

Pregunta 2 (1 punto)

Para que Git identifique quién realiza los cambios en un repositorio, es necesario configurar el nombre de usuario y el correo electrónico mediante la terminal.

- [x] Verdadero
- [ ] Falso

Explicación. Es obligatorio configurar la identidad (nombre y correo) para que queden registrados en las autorías de los commits.

Pregunta 3 (1 punto)

¿Qué comando se utiliza en la terminal para verificar la versión de Git instalada en tu computadora?

Respuestas aceptadas: git --version

Explicación. El comando git --version muestra la versión instalada actualmente en el sistema.

Pregunta 4 (2 puntos)

Relaciona cada concepto con su descripción correspondiente en el contexto de Git.

Elementos	Opciones
Control de versiones	Sistema que registra los cambios en los archivos a lo largo del tiempo.
Instantánea (Snapshot)	Estado del proyecto guardado por Git en un momento específico.

Explicación. El control de versiones evita los nombres manuales repetitivos y Git utiliza un modelo distribuido basado en instantáneas.

Módulo 2: Trabajando de forma local con Git

Aprende los comandos esenciales para registrar cambios en tus archivos y gestionar el historial del proyecto.

Creando tu primer repositorio y guardando cambios con commit (20 min)

Introducción a los repositorios locales

En esta lección aprenderás qué es un repositorio de Git y cómo transformar una carpeta común de tu ordenador en un proyecto vigilado por el control de versiones. Un repositorio local es simplemente un directorio que contiene todos los archivos de tu proyecto junto con una carpeta oculta que almacena el historial completo de cambios.

Este material formativo tiene un enfoque general y práctico. Para iniciar este camino, el primer comando fundamental que debes conocer es **git init**, el cual inicializa un repositorio vacío en tu espacio de trabajo y le indica a Git que comience a supervisar los archivos.

Preparando los cambios con git add

Una vez que has creado o modificado archivos en tu proyecto, Git no guarda los cambios automáticamente. Debes indicarle de forma explícita qué archivos deseas incluir en la próxima fotografía o versión del código. Este paso intermedio se realiza utilizando el área de preparación o *staging area*.

Para añadir un archivo específico al área de preparación, utilizas el comando `git add nombre_archivo`. Si deseas preparar todos los archivos nuevos o modificados de una sola vez, puedes utilizar el comando abreviado `git add .`. Este proceso te permite seleccionar qué modificaciones formarán parte del siguiente registro histórico.

Guardando los cambios con git commit

Una vez que los archivos están en el área de preparación, el siguiente paso es guardarlos de forma permanente en el historial del repositorio mediante un commit. El comando para realizar esta acción es `git commit -m "Mensaje descriptivo"`.

Ejemplo práctico: Imagina que has creado un archivo de texto llamado `index.html`. Para guardarlo ejecutas:

```
git init
git add index.html
git commit -m "Añadir estructura inicial de index.html"
```

El mensaje del commit debe ser corto pero claro, explicando siempre qué cambios se han realizado para facilitar futuras consultas en el historial del proyecto.

Ideas clave sobre commits y repositorios

Para consolidar lo aprendido en esta primera parte, recuerda los siguientes conceptos fundamentales:

- **git init:** Convierte una carpeta normal en un repositorio Git local.
- **git add:** Prepara los archivos seleccionados para el siguiente guardado.
- **git commit:** Crea un registro permanente en el historial del proyecto acompañado de un mensaje explicativo.

Trabajar con estos tres comandos de forma ordenada constituye el ciclo básico de trabajo diario cuando desarrollas software de manera local.

Consultando el estado y el historial del proyecto (15 min)

La importancia de conocer el estado actual

Cuando trabajas en un proyecto, es muy común olvidar qué archivos has modificado, cuáles están preparados para el commit o si existen cambios pendientes de guardar. Para resolver esta incertidumbre, Git ofrece una herramienta de consulta rápida muy potente llamada **git status**.

Este comando te muestra de forma visual y clara la situación actual de tu espacio de trabajo, indicándote los ficheros que no están siendo rastreados, los modificados y los que ya se encuentran listos en el área de preparación.

Revisando el pasado con git log

A medida que realizas múltiples commits a lo largo del tiempo, se va construyendo una línea temporal con la evolución de tu software. Para examinar este registro histórico y ver quién hizo los cambios y cuándo, utilizas el comando `git log`.

Este comando despliega una lista cronológica inversa de todos los commits realizados en el repositorio actual. Cada entrada muestra un código identificador único (hash), el autor, la fecha y el mensaje descriptivo asociado a dicho cambio, permitiéndote auditar el desarrollo del proyecto paso a paso.

Resumen de comandos de consulta

El control efectivo de un repositorio local no solo implica guardar información, sino saber consultarla en cualquier momento. Repasa las utilidades principales de esta lección:

- **git status:** Informa sobre el estado actual de los archivos en tu directorio de trabajo y área de preparación.
- **git log:** Muestra el historial completo de commits realizados en el repositorio.

Dominar estas consultas te aportará total seguridad y visibilidad sobre el estado de tus proyectos informáticos desde la terminal.

Actividades

Mi primer flujo local en Git

Abre tu terminal y realiza los siguientes pasos prácticos en una carpeta nueva de tu ordenador:

1. Crea una carpeta llamada **proyecto-prueba** y accede a ella.
2. Inicializa un repositorio de Git en su interior utilizando el comando correspondiente.
3. Crea un archivo de texto llamado **notas.txt** que contenga una breve descripción sobre ti.

4. Comprueba el estado del repositorio para identificar que el archivo no está rastreado.
5. Prepara el archivo para el commit y luego realiza el commit con el mensaje *"Primer commit del archivo de notas"*.
6. Consulta el historial del proyecto para verificar que el commit se ha registrado correctamente.

Evaluación: Comprueba lo aprendido: Trabajando de forma local con Git

Para aprobar: 60 %

Pregunta 1 (1 punto)

¿Qué comando se utiliza para inicializar un repositorio de Git en una carpeta local existente?

- [ ] git start
- [x] git init
- [ ] git new
- [ ] git create

Explicación. El comando git init es el encargado de convertir un directorio común en un repositorio Git local con su historial oculto.

Pregunta 2 (1 punto)

El comando **git status** sirve para consultar el estado actual de los archivos en el directorio de trabajo y en el área de preparación.

- [x] Verdadero
- [ ] Falso

Explicación. La afirmación es verdadera. Git status informa exactamente sobre los cambios modificados, pendientes de preparar o no rastreados.

Pregunta 3 (2 puntos)

Relaciona cada comando de Git con su función principal correspondiente:

Elementos	Opciones
git init	Inicializa un repositorio local vacío
git add	Pasa los cambios al área de preparación
git commit	Guarda de forma permanente los cambios en el historial
git log	Muestra el historial cronológico de commits

Explicación. Cada comando tiene un propósito específico dentro del flujo básico: init crea el repositorio, add prepara los archivos, commit guarda los cambios y log muestra el historial.

Pregunta 4 (1 punto)

¿Qué comando de Git se utiliza para mostrar el historial cronológico de los commits realizados en el proyecto?

Respuestas aceptadas: git log

Explicación. El comando correcto es git log.

Módulo 3: Llevando tu código a la nube con GitHub

Conecta tus proyectos locales con GitHub para respaldar tu trabajo y compartirlo con otros desarrolladores.

Introducción a GitHub y vinculación de repositorios remotos (15 min)

¿Qué es GitHub y para qué sirve?

En esta lección daremos el salto del desarrollo local a la nube utilizando **GitHub**. Hasta ahora has aprendido a gestionar tus archivos en tu propia computadora mediante Git, pero ¿qué ocurre si tu disco duro falla o quieres trabajar en equipo? Aquí es donde entra en juego GitHub.

GitHub es una plataforma basada en web que aloja repositorios Git. Funciona como un punto de encuentro centralizado donde puedes almacenar copias de seguridad de tus proyectos, mostrarlos al mundo y colaborar con otros desarrolladores de manera sencilla y ordenada. Este material formativo tiene un enfoque general y práctico para introducirte en estas herramientas profesionales.

Diferencia entre repositorio local y remoto

Para entender cómo funciona el flujo de trabajo en la nube, es fundamental distinguir entre dos conceptos clave:

- **Repositorio local:** Es el que se encuentra almacenado físicamente en tu ordenador. Es donde realizas tus cambios diarios y ejecutas los comandos *commit*.
- **Repositorio remoto:** Es la versión de tu proyecto que vive en un servidor externo, en este caso, en los servidores de GitHub. Permite centralizar el código y acceder a él desde cualquier lugar.

Vincular ambos repositorios significa establecer un puente de comunicación directo entre tu ordenador y la nube.

Creando un repositorio en la plataforma

El primer paso práctico para llevar tu código a la nube consiste en crear un repositorio vacío en la interfaz web de GitHub. Para ello, inicias sesión, haces clic en el botón de crear un nuevo repositorio, le asignas un nombre descriptivo y decides si será público o privado.

Una vez creado, GitHub te proporciona una dirección web única (URL) que identifica ese repositorio remoto. Esta dirección será la que utilices desde tu terminal local para conectar ambos mundos mediante el comando de vinculación correspondiente.

Resumen de la vinculación remota

En este bloque hemos aprendido los conceptos fundamentales sobre el alojamiento en la nube:

- GitHub actúa como un servidor central para compartir y respaldar código basado en Git.
- Existe una clara distinción entre el entorno local (tu computadora) y el entorno remoto (la nube).
- Para conectar ambos espacios se utiliza una URL única proporcionada por GitHub al crear un

nuevo proyecto en su plataforma web.

Subiendo y descargando cambios usando push y pull (20 min)

El ciclo de sincronización en la nube

Una vez que tu repositorio local está vinculado con GitHub, el siguiente paso lógico es aprender a transferir información entre ambos entornos. Este proceso de intercambio de datos se realiza principalmente a través de dos comandos esenciales: **push** y **pull**.

Estos comandos permiten mantener tu trabajo actualizado tanto en tu equipo como en el servidor remoto, facilitando el respaldo constante y la colaboración fluida con otros programadores.

Enviando tus cambios con git push

El comando `git push` se utiliza para enviar los *commits* que has realizado en tu repositorio local hacia el repositorio remoto en GitHub. Imagina que has escrito nuevo código y lo has guardado localmente; para que otros puedan verlo o para tener una copia de seguridad segura en la nube, ejecutas esta orden.

Al utilizarlo por primera vez en un repositorio recién vinculado, es común indicar el nombre del servidor remoto (por defecto *origin*) y la rama principal (como *main*).

Actualizando tu entorno con git pull

Por el contrario, el comando `git pull` sirve para descargar los cambios más recientes que existan en el repositorio remoto y fusionarlos automáticamente en tu repositorio local.

Esto resulta indispensable cuando trabajas en equipo, ya que otros colaboradores pueden haber añadido nuevas líneas de código a la nube mientras tú trabajabas en tu computadora. Hacer un *pull* te asegura estar siempre al día con la última versión del proyecto.

Ideas clave sobre push y pull

Repasemos los conceptos operativos básicos para interactuar con GitHub:

- Usa `git push` para subir tus commits locales al servidor remoto en la nube.
- Usa `git pull` para descargar y fusionar las actualizaciones del servidor remoto en tu equipo local.
- Mantener una rutina constante de envío y recepción de cambios evita conflictos complejos en el desarrollo de software.

Actividades

Conectando mi primer proyecto a GitHub

Imagina que tienes un proyecto local en tu computadora donde ya has realizado algunos commits previos. Sigue estos pasos conceptuales para completar tu práctica:

1. Crea un nuevo repositorio vacío en la plataforma web de GitHub y copia su dirección URL.
2. Abre tu terminal en la carpeta del proyecto local y vincula tu repositorio local con la URL obtenida de GitHub.

3. Envía tus commits locales hacia la nube utilizando el comando correspondiente para subir la información.

Recuerda que este ejercicio es una simulación formativa general para afianzar los conceptos estudiados sobre control de versiones remoto.

Evaluación: Comprueba lo aprendido: Llevando tu código a la nube con GitHub

Para aprobar: 60 %

Pregunta 1 (1 punto)

¿Cuál es la función principal del comando `git push`?

- [ ] Descargar los cambios recientes desde la nube.
- [x] Enviar los commits locales hacia el repositorio remoto.
- [ ] Crear un nuevo repositorio en la computadora.
- [ ] Vincular el proyecto con una dirección URL.

Explicación. El comando git push se utiliza específicamente para enviar los commits locales hacia el repositorio remoto en la nube.

Pregunta 2 (1 punto)

El repositorio remoto es la versión de tu proyecto que se encuentra alojada en los servidores de GitHub.

- [x] Verdadero
- [ ] Falso

Explicación. Es verdadero. El repositorio remoto vive en la nube (en plataformas como GitHub), mientras que el local está en tu ordenador.

Pregunta 3 (1 punto)

Escribe el comando de Git que se utiliza para descargar y fusionar los cambios de la nube en tu equipo local.

Respuestas aceptadas: git pull

Explicación. El comando correcto es git pull.

Pregunta 4 (2 puntos)

Relaciona cada concepto o comando con su descripción correspondiente:

Elementos	Opciones
Repositorio local	Versión del proyecto almacenada en tu computadora
git push	Envía los commits hacia el servidor remoto
git pull	Descarga y fusiona actualizaciones de la nube

Explicación. Cada término se corresponde directamente con su función principal dentro del flujo de trabajo con Git y GitHub.